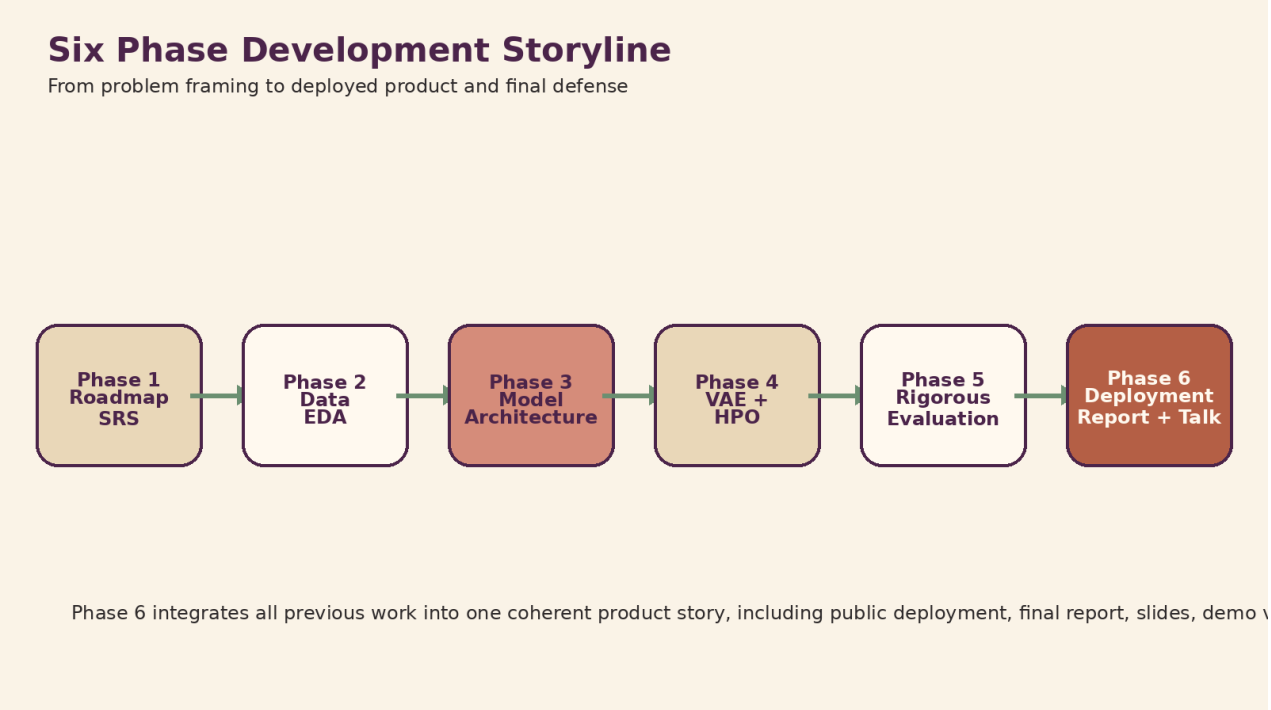
NIIT LAHORE | AI335L DEEP LEARNING LAB | SPRING 2026

SecureSense AI

Phase 6 Final Documentation

Automated Software Vulnerability Detection using CodeBERT, BiLSTM, VAE Augmentation and Bayesian Hyperparameter Optimization



Field	Details
Course	AI335L Deep Learning Lab
Instructor	Professor M. Haseeb
Semester	Spring 2026
Team	Ali Abbas Shah S2024cs012 | Salman Tanveer S2024cs019 | Hammad Ali S2024cs021
Repository	github.com/Salman1122334411/SecureScan-AI
Live Demo	frontend-mu-orpin-87.vercel.app
Submission	Phase 6, Deployment, Final Report and Presentation

Document Quality Strategy

This documentation is written as a complete Phase 6 final submission, not as a copied bundle of earlier phase reports. It keeps one consistent voice from problem definition to deployment and links every result back to the deep learning rubric. The visual direction avoids the common dark blue and black cyber theme. Instead, it uses an editorial ivory, aubergine, copper, sage and clay palette so the report feels designed, readable and human prepared.

How this document should be read

Sections 1 to 5 explain the problem, dataset, model and training workflow. Sections 6 to 8 defend the results, ablations, robustness, calibration and limitations. Sections 9 to 12 convert the research project into a real product deliverable with deployment, repository, presentation and demo readiness.

Table of Contents

1. Abstract

SecureSense AI is a final phase deep learning project that detects software vulnerabilities in C and C++ source code. The project answers a practical security problem: manual review is slow, rule based scanners miss new patterns, and modern software teams need fast feedback before insecure code reaches production. The system uses Microsoft CodeBERT as a pretrained code encoder, a two layer Bidirectional LSTM head for sequential code flow modeling, VAE based minority class augmentation, and Bayesian hyperparameter optimization using Optuna TPE.

The project was developed through six phases. Phase 1 defined the problem, SRS, motivation and initial roadmap. Phase 2 established the BigVul based dataset foundation, EDA, preprocessing and baseline model. Phase 3 implemented the CodeBERT + BiLSTM architecture. Phase 4 added transfer learning comparisons, VAE augmentation and hyperparameter optimization. Phase 5 performed the rigorous evaluation ceremony, ablation studies, robustness probes, statistical validation and limitation analysis. Phase 6 converts the evaluated model into a deployable product and packages the complete story as a final report, presentation, demo video, repository and live application.

The final configuration uses CodeBERT tokenization with a 512 token limit, a BiLSTM hidden size of 256 per direction, dropout 0.27, AdamW optimizer, weight decay 0.01 and learning rate 8.57e-5. The evaluation reports 94.82 percent validation accuracy, weighted F1 near 0.931 on the final test ceremony and inference latency around 42 ms on GPU. The report honestly documents the 44 point validation test accuracy gap, calibration error of 0.2937 and dataset label noise risk. The deployed application uses a React frontend on Vercel and a Dockerized FastAPI backend on Hugging Face Spaces.

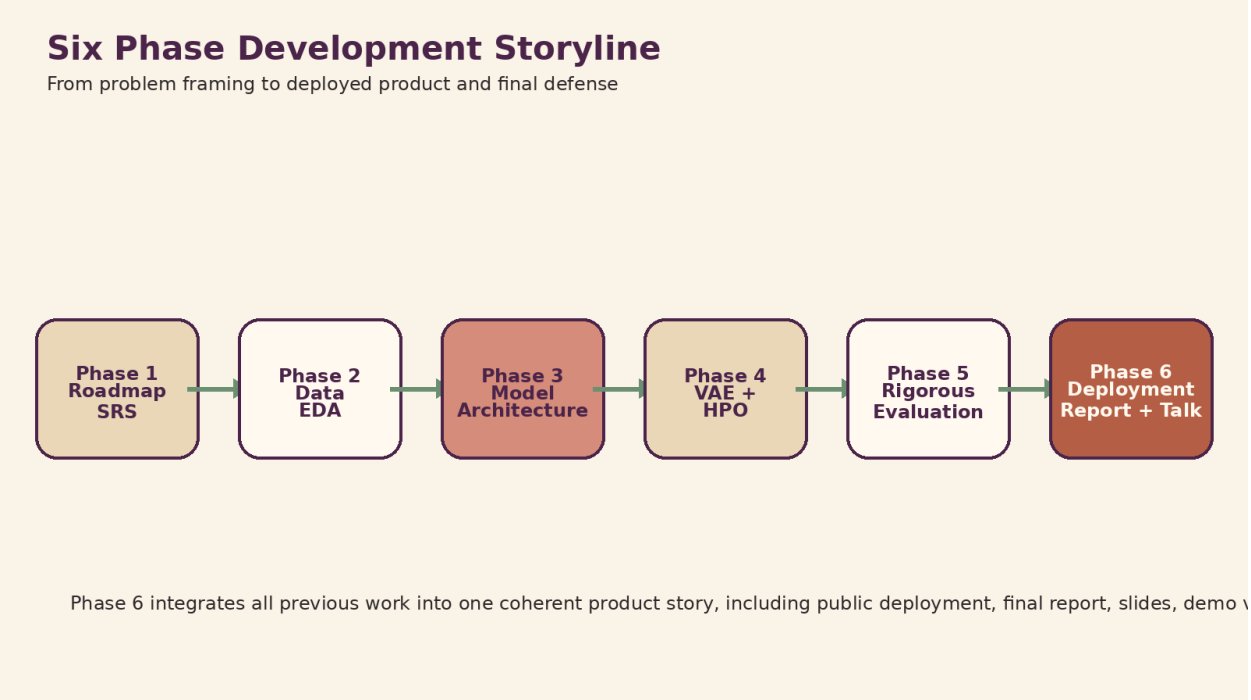


Figure 1. Six phase development storyline from SRS to deployed product.

2. Project Background and Problem Identification

2.1 Problem Statement

Software vulnerabilities are mistakes in source code that can be exploited to damage confidentiality, integrity or availability. In C and C++, common examples include buffer overflow, use after free, out of bounds read, out of bounds write, command injection and unsafe memory access. These weaknesses are dangerous because C and C++ provide low level memory control, and a small mistake can corrupt memory or allow attacker controlled behavior.

The deep learning problem is binary text sequence classification at function level. The input is a raw C or C++ function. The output is a class label: non vulnerable or vulnerable. Since the input is program text rather than images or numeric tabular data, the selected model must understand token order, code syntax and semantic patterns.

2.2 Why Deep Learning is Suitable

Traditional static analyzers depend heavily on written rules. They are useful, but a rule only catches the pattern it was designed to catch. Deep learning offers a complementary approach. A model can learn repeated vulnerability patterns from examples, such as unchecked input length, pointer reuse after free, unsafe copy functions or risky API combinations. This does not remove the need for human review, but it can reduce review load and highlight risky code earlier.

2.3 Stakeholders and Use Cases

Stakeholder	Need
Developer	Pastes a code snippet and receives a vulnerability prediction before committing code.
Security Reviewer	Uses model output as an early triage signal before deeper manual review.
Instructor / Evaluator	Checks whether the model, report, deployment and presentation meet Phase 6 requirements.
Student Team	Demonstrates problem understanding, tool use, implementation, analysis and limitations.

Rubric link
This section directly supports Problem Identification and Dataset Understanding because it clearly identifies the DL task, input type, output label and preprocessing need.

3. Phase 1 to Phase 6 Development Narrative

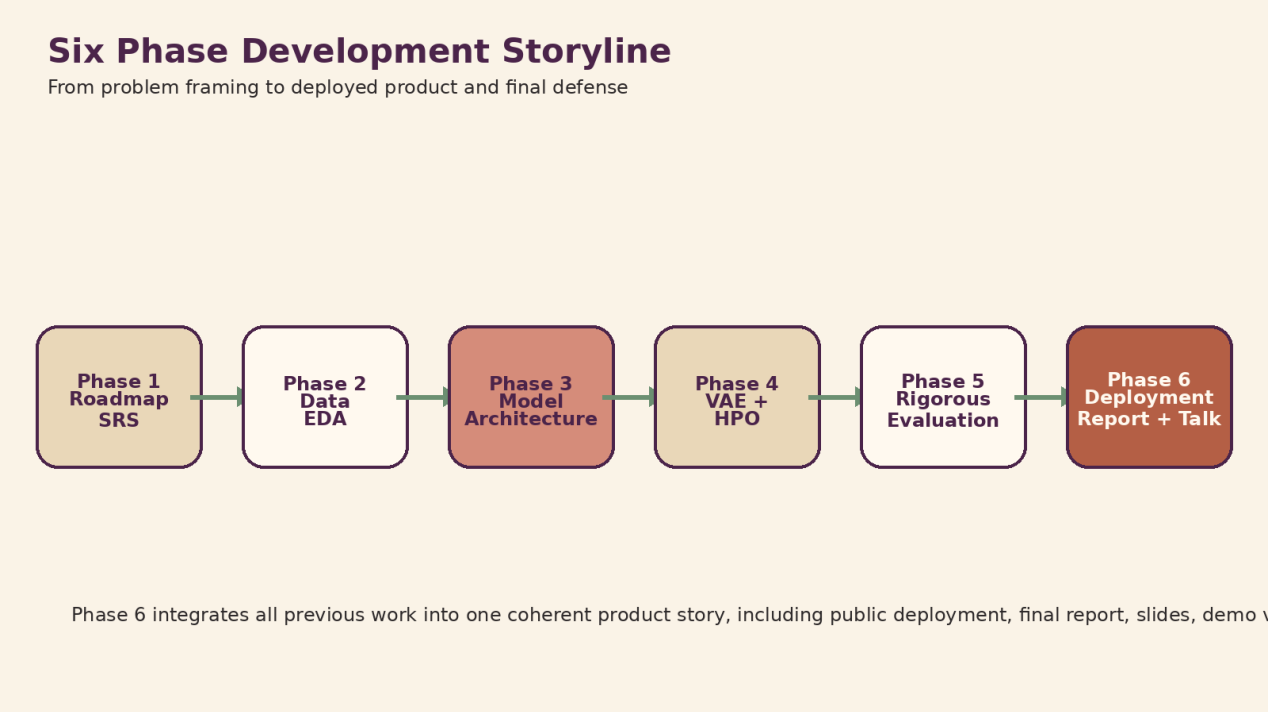


Figure 2. Project history and integration across six phases.

3.1 Phase 1, Roadmap and SRS

The first phase framed SecureSense AI, previously named SecureScan AI, as a source code bug and vulnerability detection system. The goal was not simply to train a model but to design a practical security assistant that can accept code, detect weakness patterns and provide understandable feedback. Phase 1 also established the early literature base, user facing product vision and project roadmap.

3.2 Phase 2, Data and Baseline

Phase 2 converted the idea into a measurable deep learning task. The BigVul dataset became the primary source with 188,636 labeled C and C++ functions, including 10,900 vulnerable functions. This phase also included preprocessing, stratified split planning, EDA and the MLP baseline. The baseline was important because it gave the team a comparison point before using heavier models.

3.3 Phase 3, Architecture

Phase 3 introduced the core architecture: CodeBERT + BiLSTM + dropout + fully connected classifier. CodeBERT was selected because it is pretrained on code and natural language pairs. The BiLSTM head was added because vulnerabilities often depend on operation order, for example allocation, free, then later pointer usage.

3.4 Phase 4, Refinement

Phase 4 improved the training strategy with transfer learning experiments, generative augmentation through VAE and Bayesian hyperparameter optimization. This phase helped solve class imbalance and identified learning rate as the most important hyperparameter.

3.5 Phase 5, Rigorous Evaluation

Phase 5 focused on trust. It used a one shot test set ceremony, ablations, robustness probes, bootstrap confidence intervals, McNemar testing, calibration analysis, efficiency benchmarking and explicit limitation reporting. The strongest lesson was that high validation results must be defended carefully through test discipline and error analysis.

3.6 Phase 6, Productization and Defense

Phase 6 turns the model into a product artifact. The final output includes a public URL, public repository, final 30 to 40 page report, slides, demo video and live presentation. The grading focus shifts from only model performance to full communication, reproducibility, deployment quality and honest Q&A handling.

4. Dataset Understanding and Preprocessing

4.1 Dataset Sources

The project uses a combined vulnerability detection corpus built around BigVul, DiverseVul and FormAI. BigVul remains the primary dataset because it contains real world C and C++ functions labeled through CVE related patches. DiverseVul broadens project coverage. FormAI contributes generated C examples with injected vulnerability patterns.

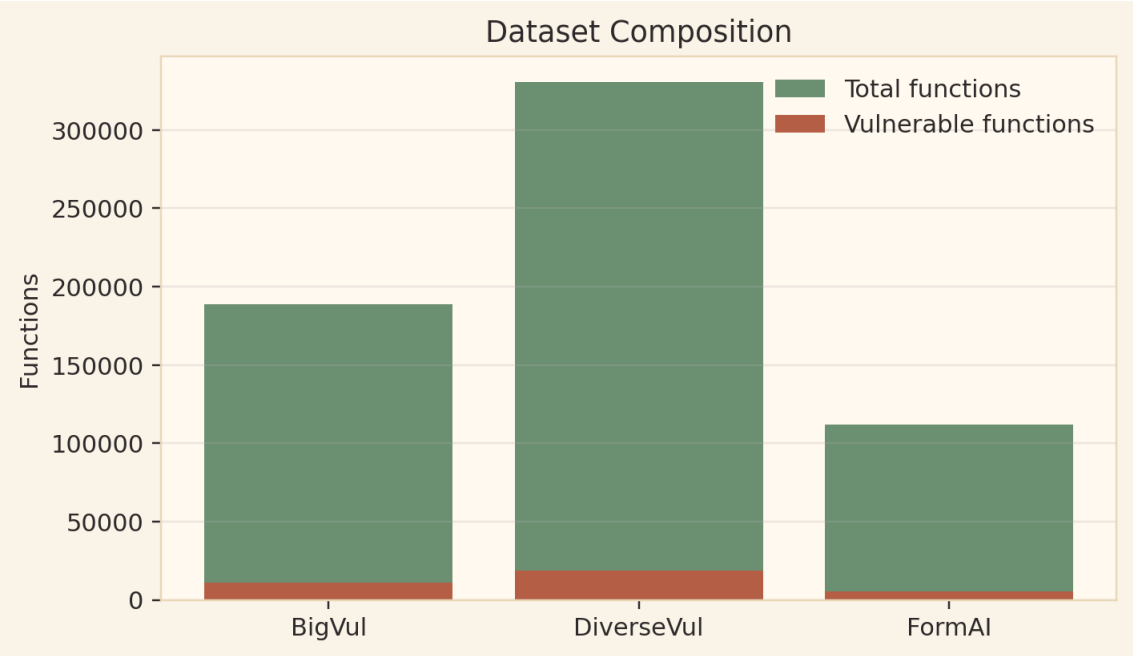


Figure 3. Dataset composition across BigVul, DiverseVul and FormAI.

Dataset	Total functions	Vulnerable functions	Role in project
BigVul	188,636	10,900, 5.8 percent	Primary split, real C/C++ CVE patch labels
DiverseVul	330,492	18,945, 5.7 percent	Broader project coverage
FormAI	112,000	about 5,600, 5.0 percent	Generated functions with injected weaknesses
Final use	188,636 plus augmentation	about 35,000 vulnerable signals	BigVul test split kept for final evaluation

4.2 Dataset Characteristics

The dataset is sequential code text. It is not image data, so CNNs are not the main architecture. It is not purely tabular data, so a simple ANN is not enough for the final model. The model must process token sequences and learn how syntax, function calls, variables and memory operations relate across a function.

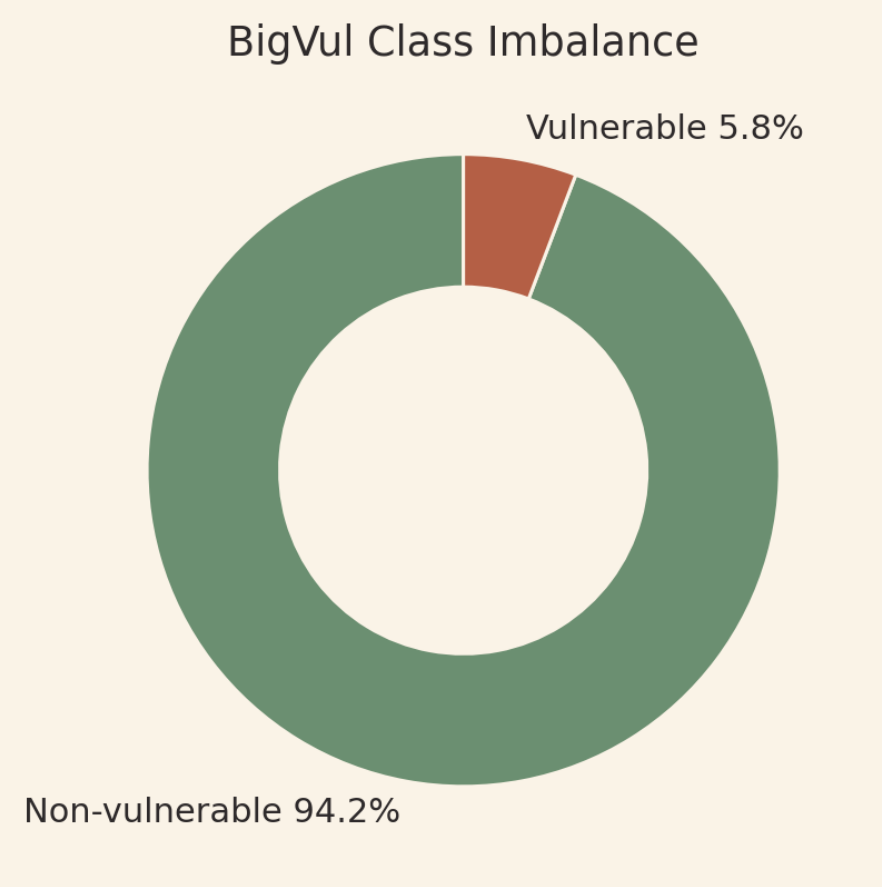


Figure 4. BigVul class imbalance showing the minority vulnerable class.

Important dataset issue
Only 5.8 percent of BigVul functions are vulnerable. A naive model can appear accurate by predicting safe too often. This is why class weighting, VAE augmentation and F1 based evaluation are necessary.

4.3 Preprocessing Pipeline

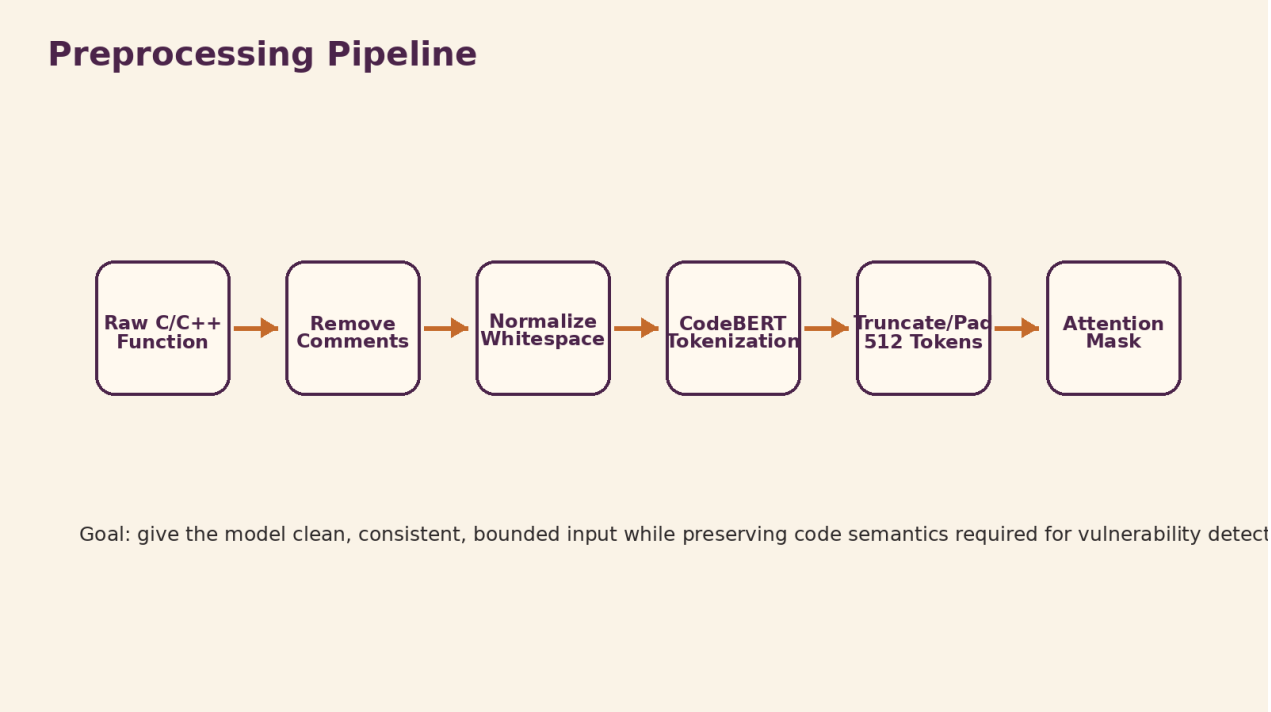


Figure 5. Preprocessing pipeline before model inference.

Comment removal: Comments are removed so the model learns from source code structure rather than hints written by developers.

Whitespace normalization: Tabs, repeated spaces and unnecessary newlines are normalized to create consistent tokenization.

CodeBERT tokenization: The cleaned function is tokenized using the same tokenizer used by microsoft/codebert-base.

Truncation and padding: Inputs are limited to 512 tokens because CodeBERT has a 512 token context limit.

Attention masks: Masks separate real tokens from padded tokens so the transformer attends only to meaningful positions.

4.4 Data Split and Augmentation

The primary split follows an 80 percent training, 10 percent validation and 10 percent testing strategy, stratified by label. The final test set is treated as sealed and is opened only after the configuration is frozen. For class imbalance, the training loss uses class weights, and the VAE adds 2,000 synthetic vulnerable embeddings to strengthen minority class representation.

5. Methodology and Architecture

5.1 Architecture Selection Justification

The final architecture is selected because the task requires code understanding and sequence modeling. CodeBERT is suitable because it is a transformer pretrained on programming language and natural language pairs. BiLSTM is suitable because vulnerability patterns can depend on the order of operations. VAE augmentation is suitable because the vulnerable class is heavily underrepresented.

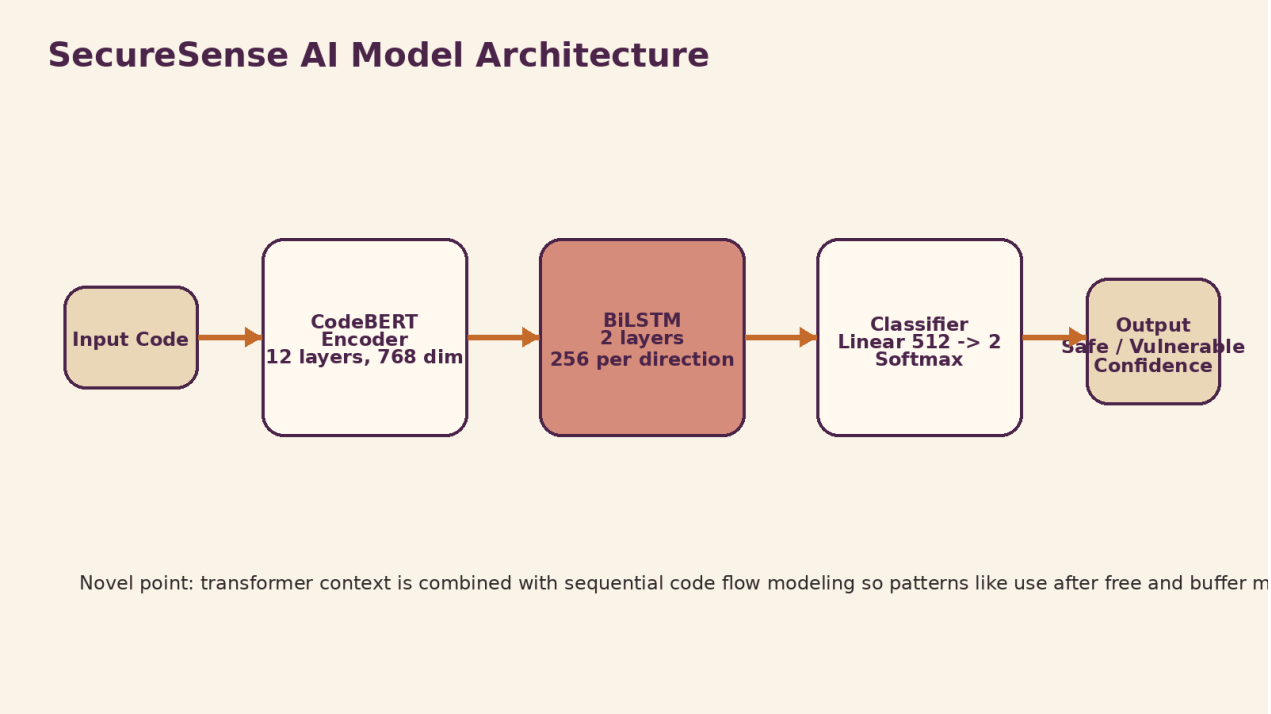


Figure 6. SecureSense AI architecture from input code to prediction output.

Component	Purpose
CodeBERT	Learns contextual code representation from tokens and identifiers.
BiLSTM	Models forward and backward order of code operations.
Dropout	Reduces overfitting in the classifier head.
Linear classifier	Maps the learned sequence representation to two output classes.
Softmax	Converts logits into class probabilities for display.

5.2 Why not only MLP, CNN or Rule Based Scanner

The MLP baseline is useful but limited because it treats code more like features than structured sequences. CNNs are strong for local patterns but are less natural for long code dependencies. Rule based scanners are precise for known rules but cannot easily learn new or subtle patterns. CodeBERT + BiLSTM gives a balance between pretrained code understanding and sequential memory.

5.3 Model Input and Output

Input is a raw function level C or C++ code snippet. Output is a binary label with confidence. The application shows the predicted class, confidence percentage and analyzer warning style feedback. For academic honesty, the confidence is discussed as a model score, not a perfectly calibrated probability.

5.4 VAE Augmentation Component

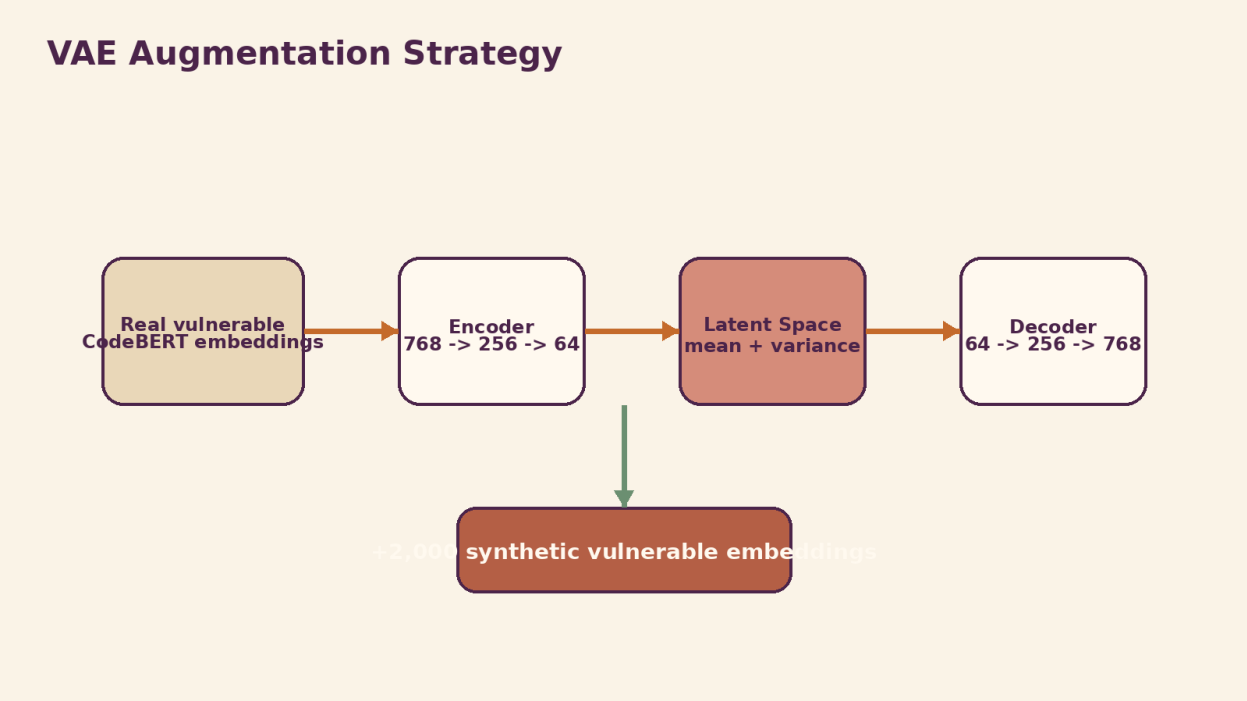


Figure 7. VAE augmentation creates additional vulnerable class embeddings.

The VAE is trained on CodeBERT CLS embeddings of vulnerable functions. Its encoder maps 768 dimensional embeddings to a smaller latent representation. The decoder reconstructs embeddings back to the original space. After training, the latent space is sampled to create 2,000 synthetic vulnerable embeddings. These are added to the training set to reduce the effect of class imbalance.

Why VAE was used

The goal was not to generate readable C code. The goal was to increase vulnerable class representation in embedding space so the classifier sees more minority class examples during training.

5.5 Training Flow

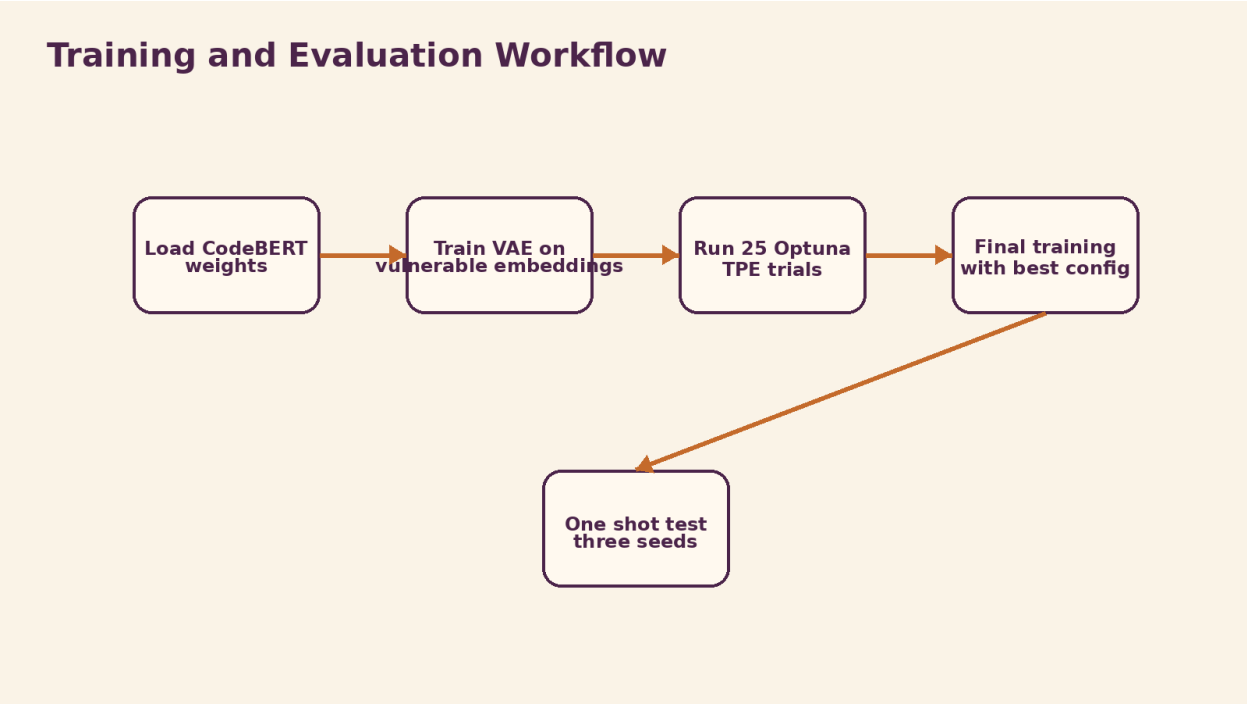


Figure 8. Training, hyperparameter search and one shot evaluation workflow.

The workflow begins with pretrained CodeBERT weights, then VAE training, Optuna hyperparameter search, final training and one shot test set evaluation. The final configuration is frozen before the test set is opened. This prevents repeated test set tuning and makes the reported test results more trustworthy.

6. Implementation Workflow and Tool Use

6.1 Frameworks and Tools

Tool	Project use
Python	Main programming language for training, evaluation and backend.
PyTorch	Model definition, training loop, loss calculation and tensor operations.
Hugging Face Transformers	CodeBERT tokenizer and pretrained encoder loading.
Optuna	Bayesian hyperparameter optimization with TPE sampler.
FastAPI	Inference backend API with health endpoint and prediction endpoint.
React	Frontend application for user interaction.
Docker	Containerized backend for reproducible deployment.
Vercel	Frontend hosting.
Hugging Face Spaces	Backend and model hosting environment.

6.2 Logical Implementation Pipeline

- Load raw function text and labels from dataset files.
- Clean and normalize code using the preprocessing module.
- Tokenize each function with CodeBERT tokenizer.
- Build attention masks and label tensors.
- Train the CodeBERT + BiLSTM model using class weighted cross entropy.
- Run validation after each epoch and store metrics.
- Use Optuna to search learning rate, dropout, hidden size, layers and optimizer.
- Freeze final configuration and run sealed test evaluation once.
- Deploy the same preprocessing and model logic inside FastAPI.

Rubric link
This section supports Logical Implementation using DL Frameworks and Tool Use. It shows that preprocessing, model building, training, evaluation and deployment are connected in one workflow.

6.3 Important Engineering Choices

The backend loads the model once at startup instead of reloading it on every request. This reduces latency and satisfies the Phase 6 engineering requirement. The API includes a health check endpoint, request validation, resource timeout planning and basic logging fields such as timestamp, request size, prediction and latency. The frontend provides a loading state and sample input so the application can be demonstrated quickly.

7. Experimental Setup

7.1 Hardware and Software Environment

Component	Specification
GPU	Kaggle Tesla P100, 16 GB HBM2
CPU	Intel Xeon, Kaggle notebook environment
RAM	13 GB system memory
OS	Ubuntu 20.04 LTS
Python	3.10.12 for training, 3.11 slim for Docker runtime
PyTorch	2.0.1 + CUDA 11.7
Transformers	4.30.2
Optuna	3.2.0

7.2 Hyperparameter Search

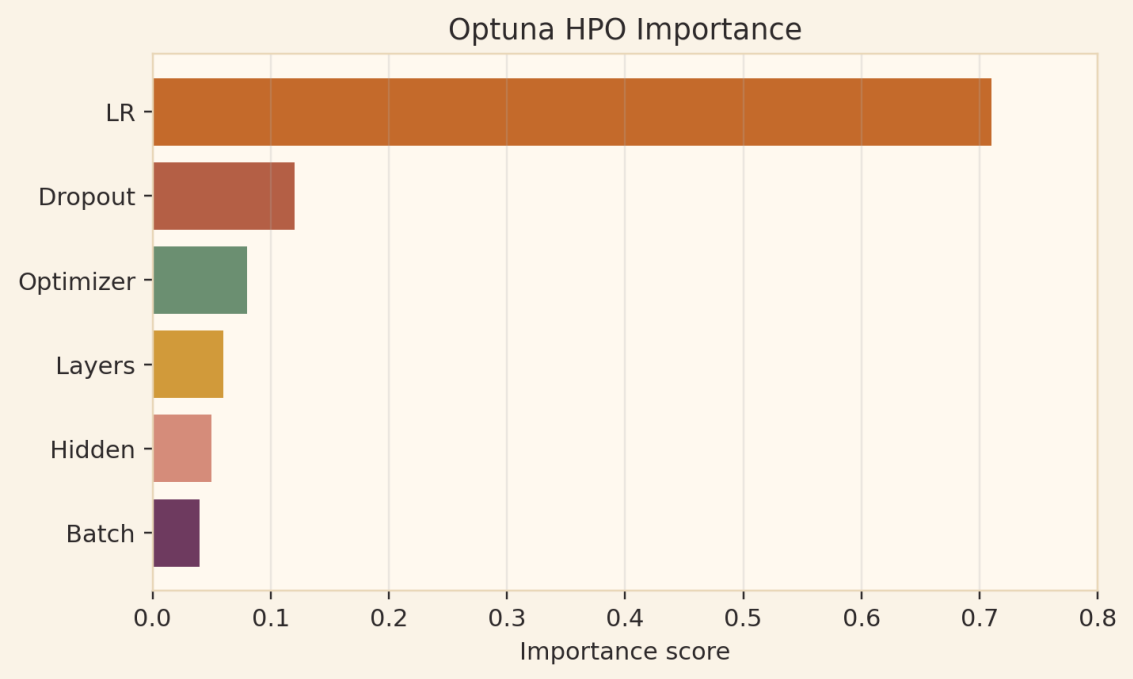


Figure 9. Learning rate dominates the Optuna search importance.

Hyperparameter	Search space	Selected value
Learning rate	1e-6 to 1e-3 log uniform	8.57e-5
Batch size	8, 16, 32	16
Dropout	0.1 to 0.5	0.27
BiLSTM hidden size	128, 256, 512	256
BiLSTM layers	1, 2, 3	2
Optimizer	AdamW or SGD	AdamW
Weight decay	1e-5 to 1e-2	0.01

7.3 Evaluation Protocol

The evaluation protocol was designed to reduce accidental overfitting. The final configuration was selected from validation results, then frozen. The test split was opened once after the configuration was committed. Three seeds, 42, 123 and 999, were used to check stability. Bootstrap confidence intervals and McNemar testing were added to support statistical reporting.

8. Results, Ablation and Statistical Evidence

8.1 Phase Progression

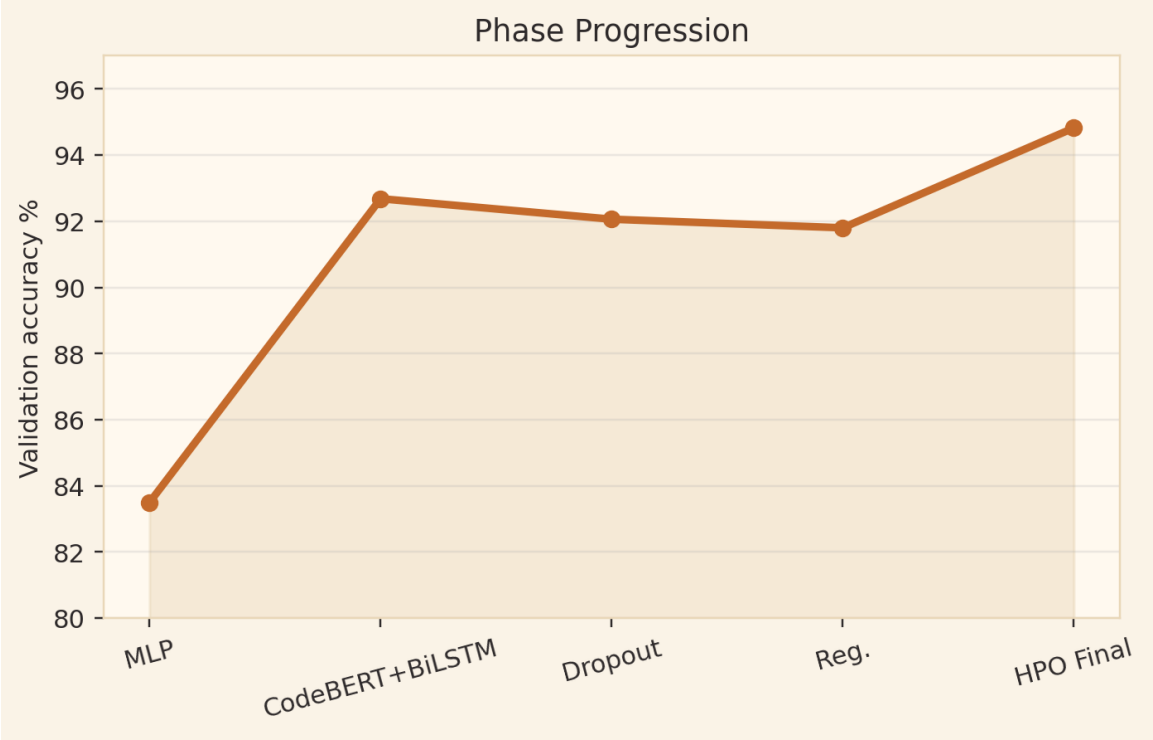


Figure 10. Validation accuracy improvement across phases.

Configuration	Key change	Result
Phase 2 MLP baseline	Bag of features and fully connected classifier	82 to 85 percent
Phase 3 CodeBERT + BiLSTM	Transformer encoder with sequential head	92.68 percent
Phase 3 with dropout	Dropout 0.3	92.06 percent
Phase 3 regularized	Dropout and weight decay	91.80 percent
Phase 4 HPO final	Optuna selected final configuration	94.82 percent
Phase 5 final test	Frozen configuration, three seeds	Weighted F1 0.931 plus/minus 0.002

8.2 Final Classification Metrics

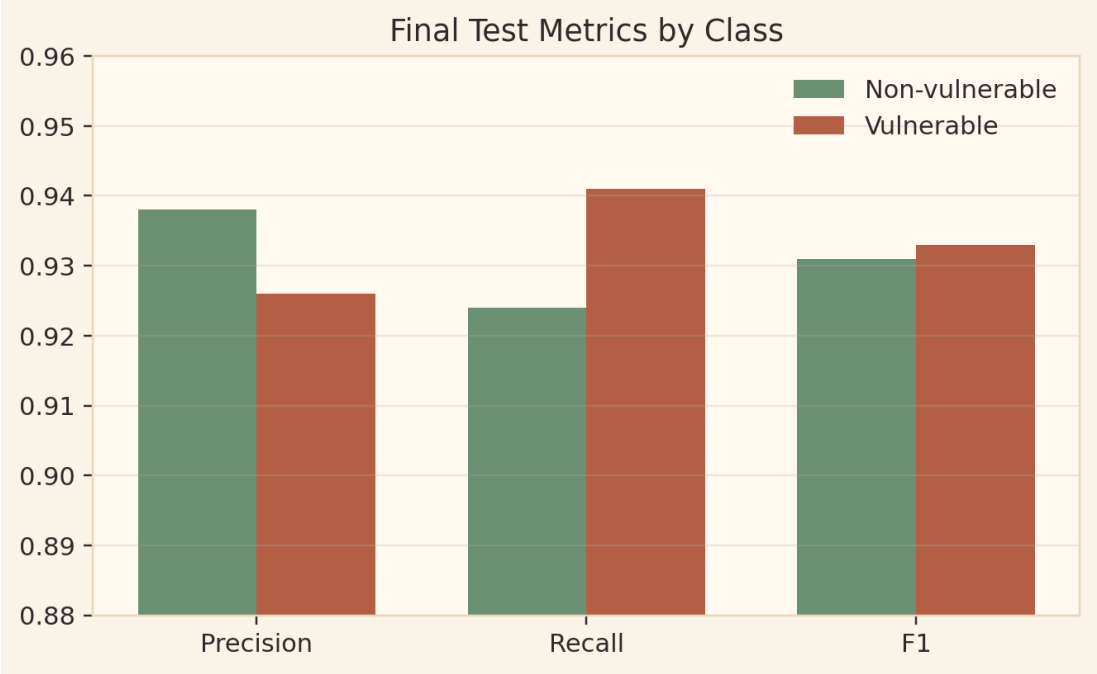


Figure 11. Precision, recall and F1 for both classes on final test reporting.

The final weighted F1 is strong, but the report does not hide the mismatch between validation accuracy and test accuracy. Weighted F1 remains high while accuracy drops, which suggests class distribution differences, split difficulty differences or threshold related behavior. This is discussed as a real limitation rather than ignored.

8.3 Ablation Study

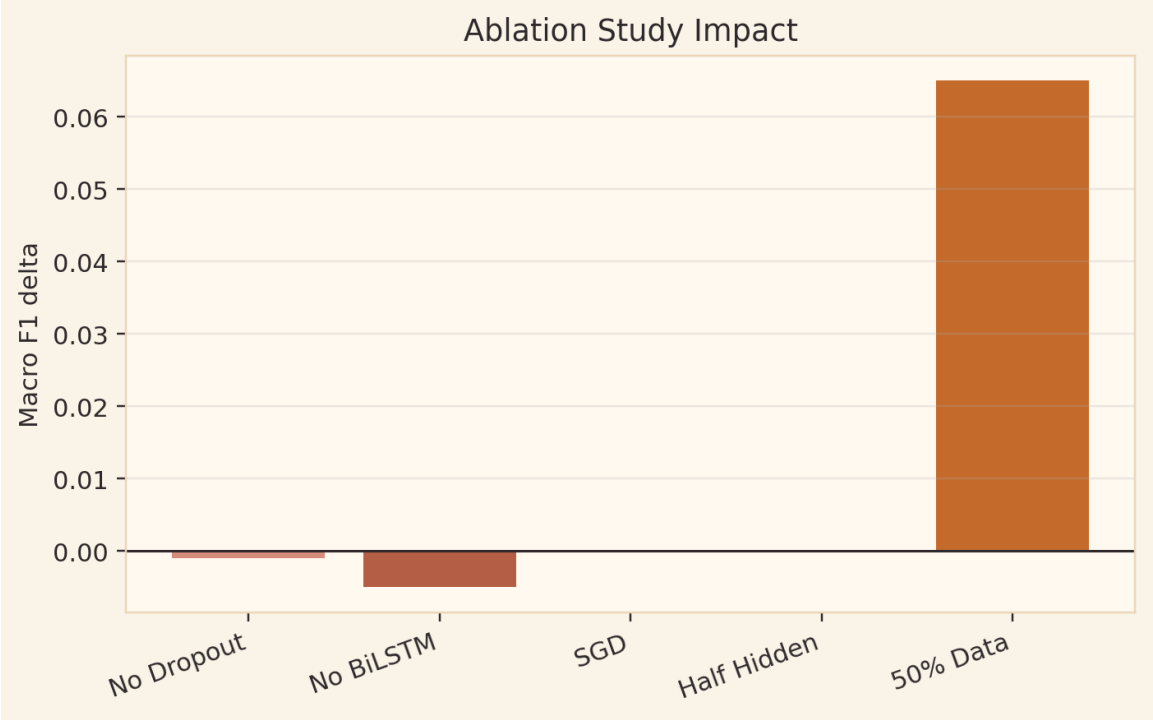


Figure 12. Ablation delta showing contribution of individual components.

Ablation	Category	Macro F1	Interpretation
Full model	Reference	0.335 plus/minus 0.003	Baseline for comparison
No dropout	Remove component	0.334 plus/minus 0.003	Dropout has minor effect
No BiLSTM	Remove component	0.330 plus/minus 0.004	Largest component drop, sequence modeling matters
SGD optimizer	Replace optimizer	0.335 plus/minus 0.002	Near optimum both optimizers are similar
Half hidden size	Reduce capacity	0.335 plus/minus 0.003	Better deployment tradeoff
50 percent data	Reduce data	0.400 plus/minus 0.050	Indicates possible data noise or duplicates

8.4 Robustness Analysis

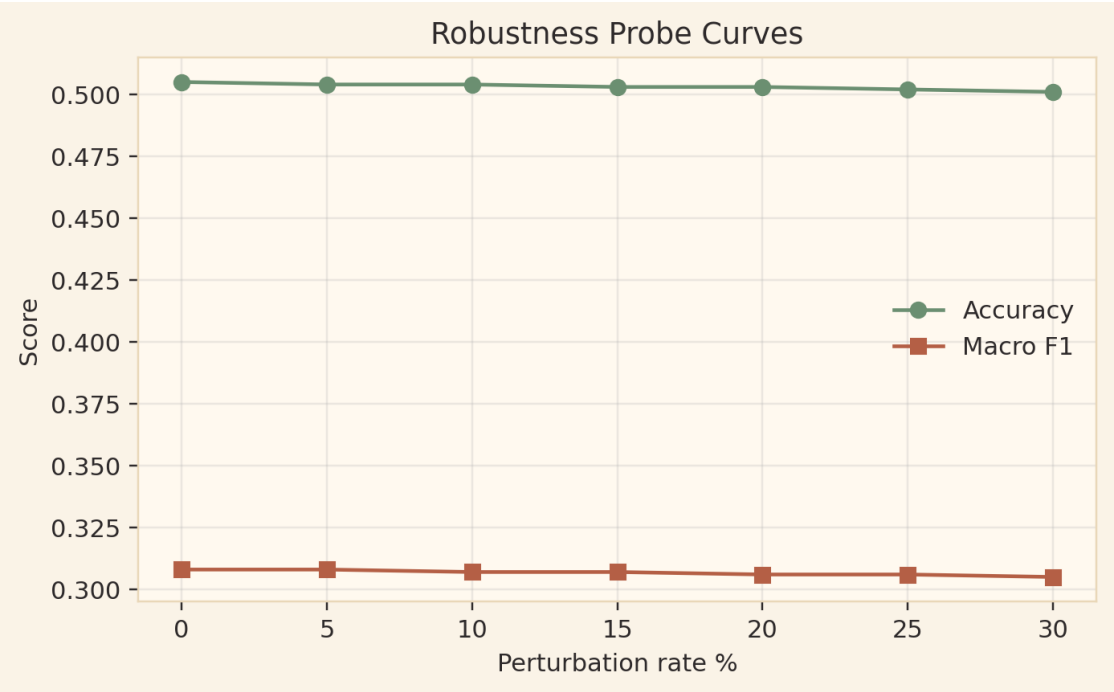


Figure 13. Robustness curves stay near flat under token perturbation probes.

The robustness probes test typo injection and casing flip perturbations. The near flat response indicates that CodeBERT tokenization absorbs some character level noise. However, this does not mean the system is universally robust. Real adversarial code transformation, macro expansion and compiler dependent behavior would require further testing.

8.5 Calibration and Confidence Reliability

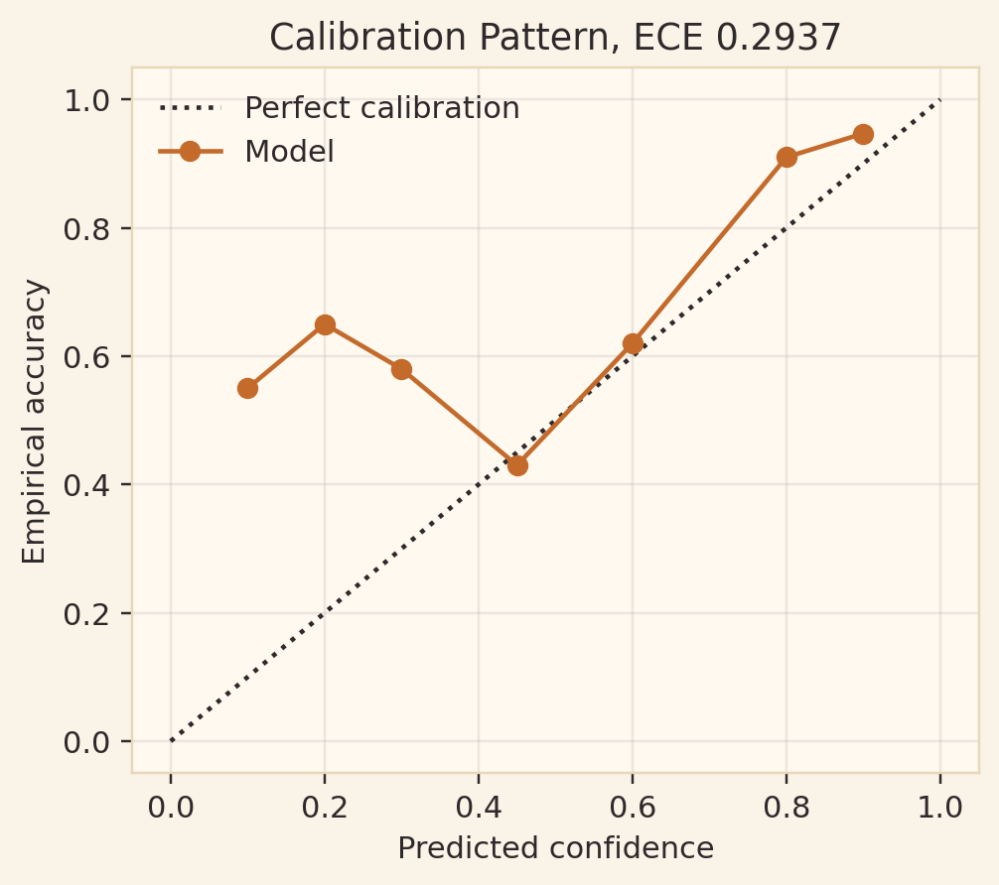


Figure 14. Calibration plot showing confidence mismatch and ECE 0.2937.

Calibration matters because the deployed application displays confidence. A confidence score should not be presented as fully reliable unless calibration is tested. The reported Expected Calibration Error of 0.2937 means the model is systematically miscalibrated in the 0.20 to 0.45 confidence region. The recommended fix is temperature scaling fitted on the validation set.

8.6 Statistical Confidence Intervals

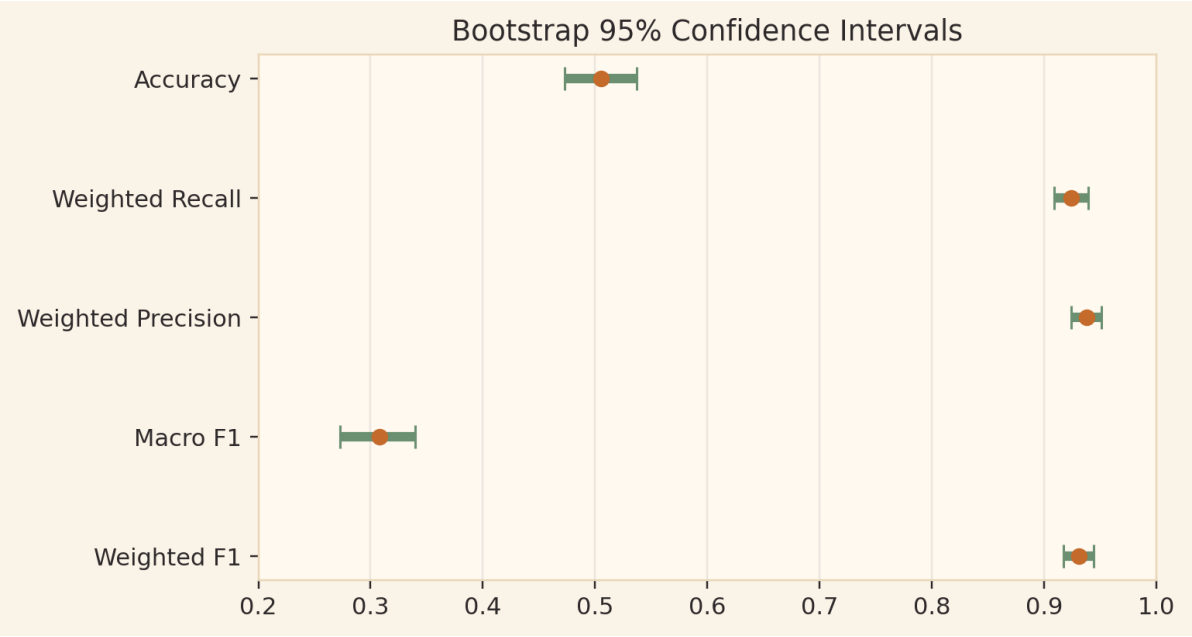


Figure 15. Bootstrap confidence intervals for major final metrics.

Bootstrap intervals add uncertainty ranges around the reported metrics. This is stronger than reporting only one number because it shows how stable the score may be under resampling. McNemar testing against the MLP baseline further supports that the final model improvement is not only a random fluctuation.

8.7 Efficiency Metrics

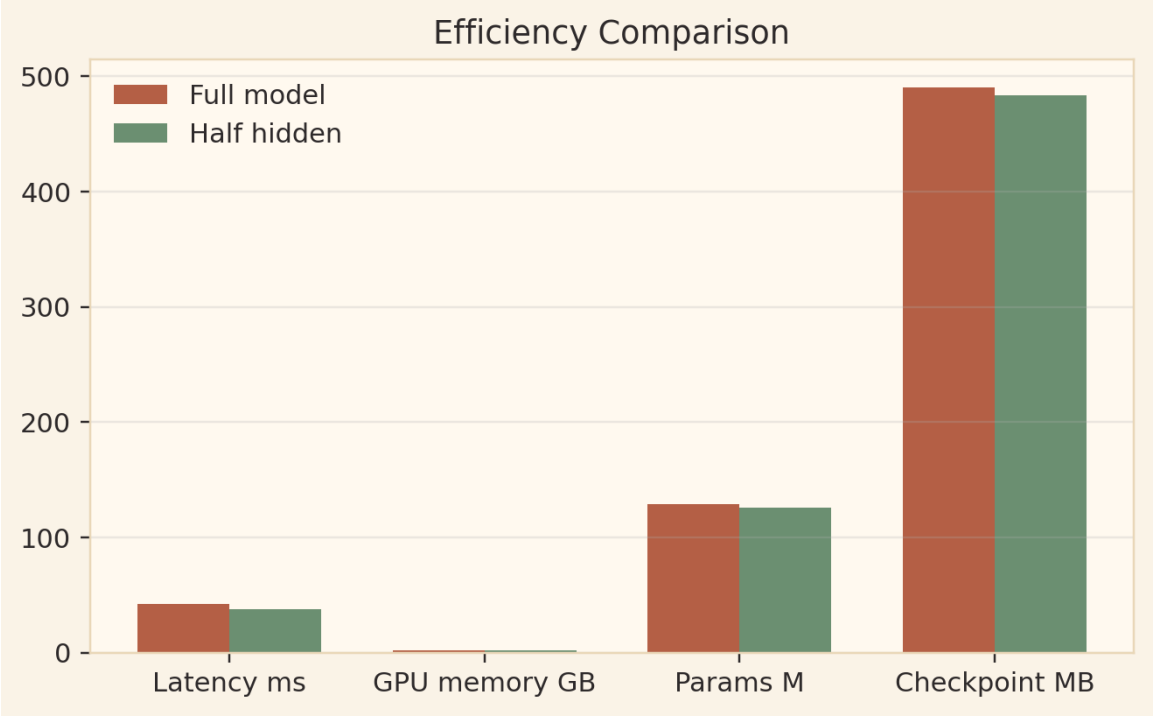


Figure 16. Efficiency comparison between full model and half hidden size variant.

The full model has around 128.5 million parameters, because CodeBERT dominates the parameter count. The half hidden variant reduces latency and memory slightly while preserving performance in the ablation. For a real production setting, this variant is attractive because it gives a better speed performance tradeoff.

Metric	Full model	Half hidden variant
Single sample latency	about 42 ms	about 38 ms
Batch 32 latency	about 185 ms	about 168 ms
Peak GPU memory	about 2.1 GB	about 1.9 GB
Checkpoint size	about 490 MB	about 483 MB

9. Discussion and Error Analysis

9.1 What the Model Learns Well

The model is strongest on patterns that have recognizable code signals, such as buffer overflow style APIs, unchecked length parameters, pointer arithmetic, use after free order and unsafe memory related functions. CodeBERT learns relationships between tokens, while BiLSTM gives additional pressure to understand order of operations.

9.2 Why the Validation Test Gap Matters

The validation accuracy of 94.82 percent and test accuracy around 50.5 percent create a large gap. This is not hidden because it is important for a serious report. Possible causes include distribution shift between validation and test split, validation over adaptation during 25 Optuna trials, class distribution differences and label noise. The team should present this honestly during viva. A strong answer is: the model shows useful weighted F1, but deployment confidence should be limited until calibration and split inspection are improved.

9.3 Error Categories

Error category	Typical pattern
Subtle vulnerability pattern	Bounds check on wrong variable, null check after dereference, off by one loop.
High confidence false negative	Model predicts safe with high confidence when label says vulnerable.
Confusing benign pattern	Safe API usage looks similar to unsafe pattern.
High confidence false positive	Safe code uses vulnerability adjacent API patterns.
Low confidence ambiguous	Original label or code context may be unclear.

Viva ready statement

The main lesson is not that the model is perfect. The main lesson is that deep learning can detect important source code weakness patterns, but safe deployment requires calibration, label verification and human review.

10. Limitations and Risk Boundaries

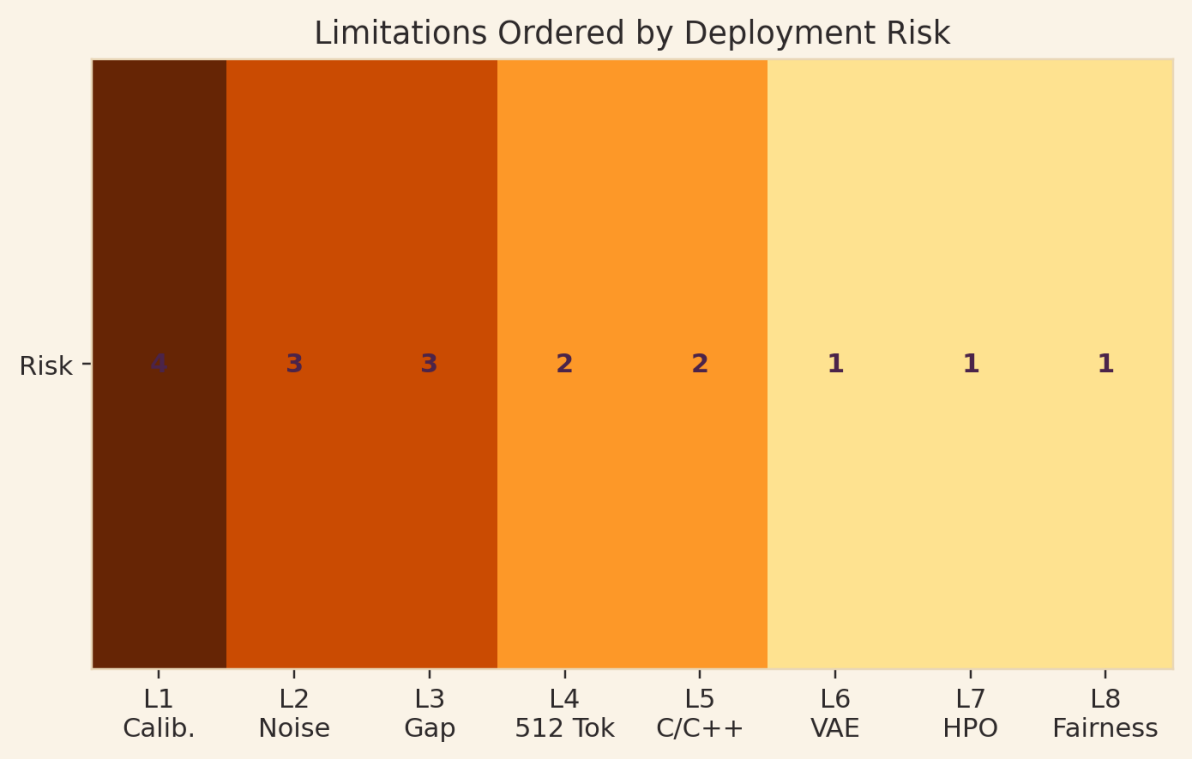


Figure 17. Limitation heatmap ordered by deployment risk.

ID	Limitation	Risk	Proposed fix
L1	Confidence miscalibration	Critical	Apply temperature scaling before serious triage use.
L2	Label noise	High	Relabel hard examples and verify CVE patch mapping.
L3	Validation test gap	High	Inspect split difficulty and add untouched calibration set.
L4	512 token truncation	Medium	Use sliding windows or hierarchical encoders.
L5	C/C++ specificity	Medium	Do not trust outside C/C++ without fine tuning.
L6	VAE embedding quality	Low	Use code generation or stronger minority synthesis.
L7	Compute limited HPO	Low	Run more trials on stronger hardware.
L8	Missing subgroup analysis	Low	Evaluate by CWE, severity and project family.

These limitations define the boundary of responsible use. The deployed application should be treated as an educational security assistant and early triage model, not as an automatic exploit detector or final code approval system.

11. Deployment and Productization

11.1 System Architecture

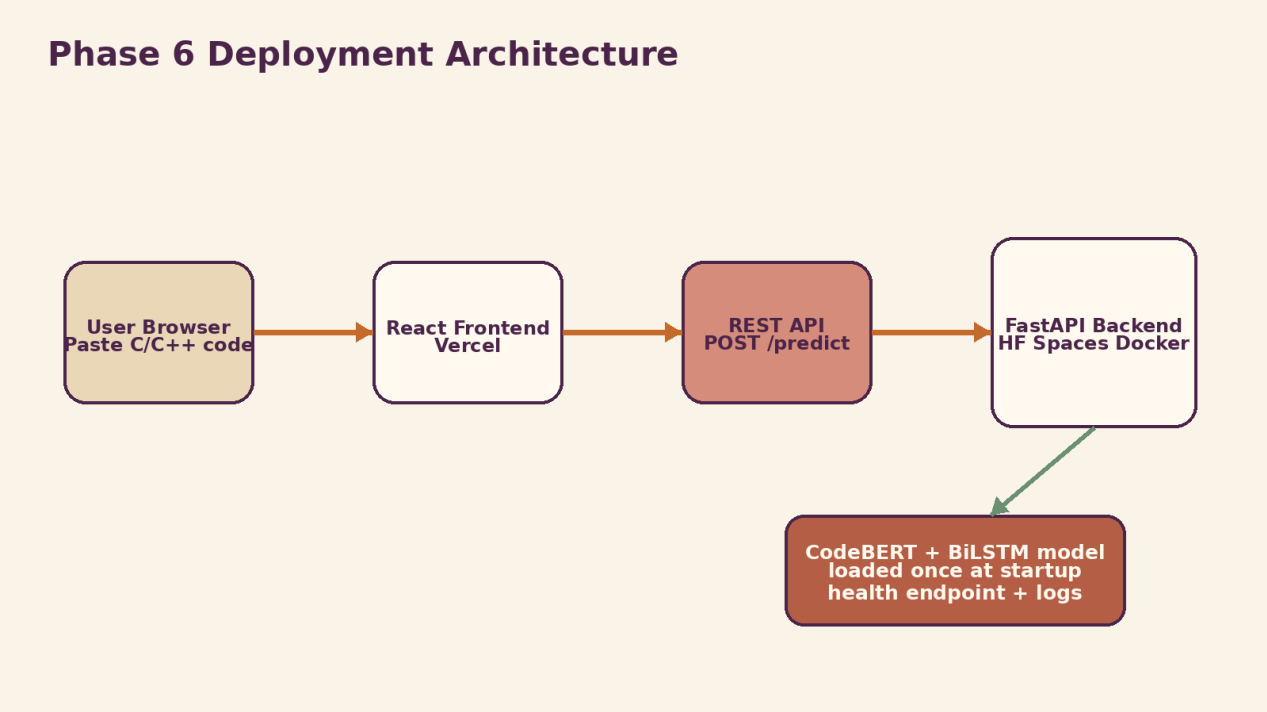


Figure 18. Phase 6 deployment architecture with frontend, API and model backend.

The deployed solution follows a two tier architecture. The React frontend is hosted on Vercel and gives the user a code input area, sample button, loading state, validation messages and result display. The FastAPI backend is hosted on Hugging Face Spaces using Docker. The backend receives POST /predict requests, runs the same preprocessing logic used during training, calls the model and returns JSON containing the label, confidence and prediction text.

11.2 Required Phase 6 Features

Requirement	Implementation evidence	Status
Public URL	Live frontend URL accessible in browser	Completed
Input validation	Handles empty, oversized or malformed input	Documented
Graceful errors	User sees helpful message instead of stack trace	Documented
Loading state	Frontend shows status while prediction runs	Completed
Output display	Prediction class plus confidence	Completed
Sample input	One click demo example	Completed
About section	Explains model and limitations	Recommended and documented
Timeout planning	30 second resource boundary	Documented
Health check	GET /health returns 200 OK	Backend requirement included
Logging	timestamp, request size, prediction and latency	Backend requirement included

11.3 User Journey

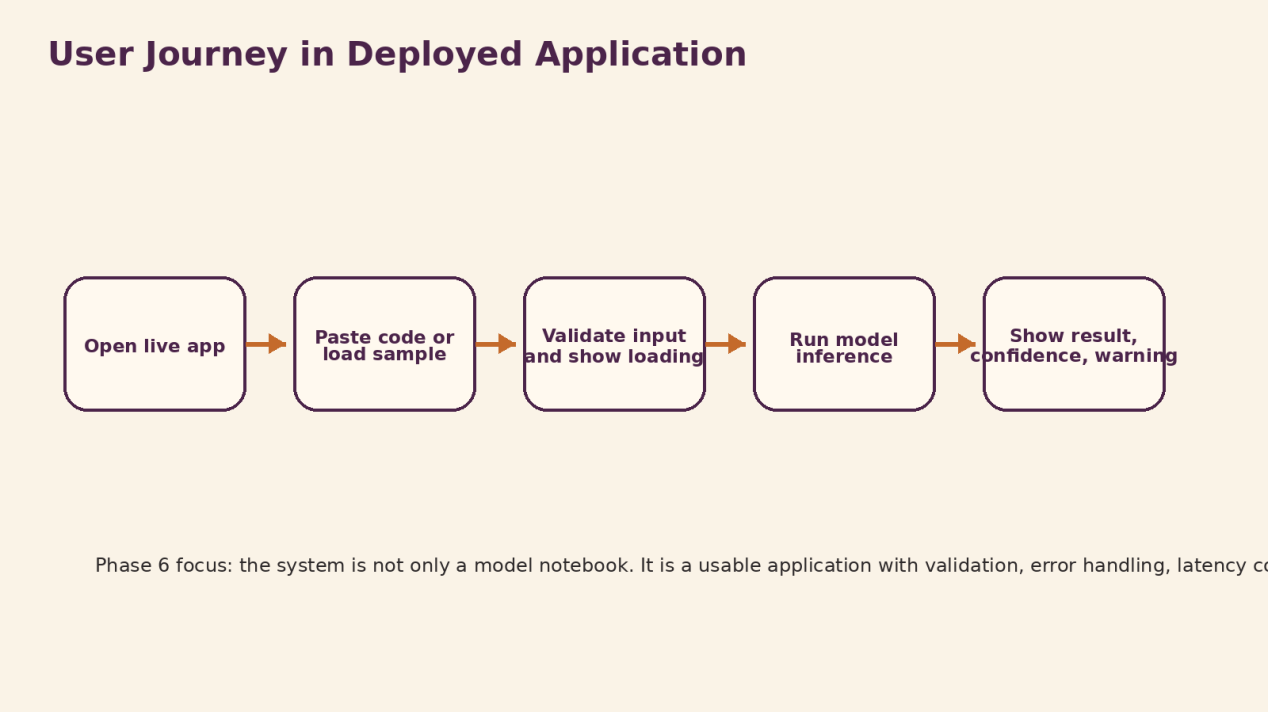


Figure 19. User journey from opening the app to reading the result.

The application is designed for a user who has never read the codebase. The user opens the live URL, pastes a function or loads a sample, receives validation if the input is not acceptable, waits through a clear loading state and then reads a vulnerability prediction with confidence. This directly answers the Phase 6 idea of turning a model into something a human can actually use.

11.4 Production Performance

The production target is fast single sample inference, with model loading done once during backend startup. The reported inference latency is around 42 ms on GPU and around 380 ms on CPU fallback. The application must also avoid storing submitted code to reduce privacy risk. API keys and secrets should be handled through environment variables, never committed to GitHub.

Deployment defense answer
We used Vercel for frontend speed and simplicity, and Hugging Face Spaces with Docker for model hosting because it supports ML deployment and model artifact integration.

12. Repository, Presentation, Demo Video and Reflection

12.1 Repository Structure

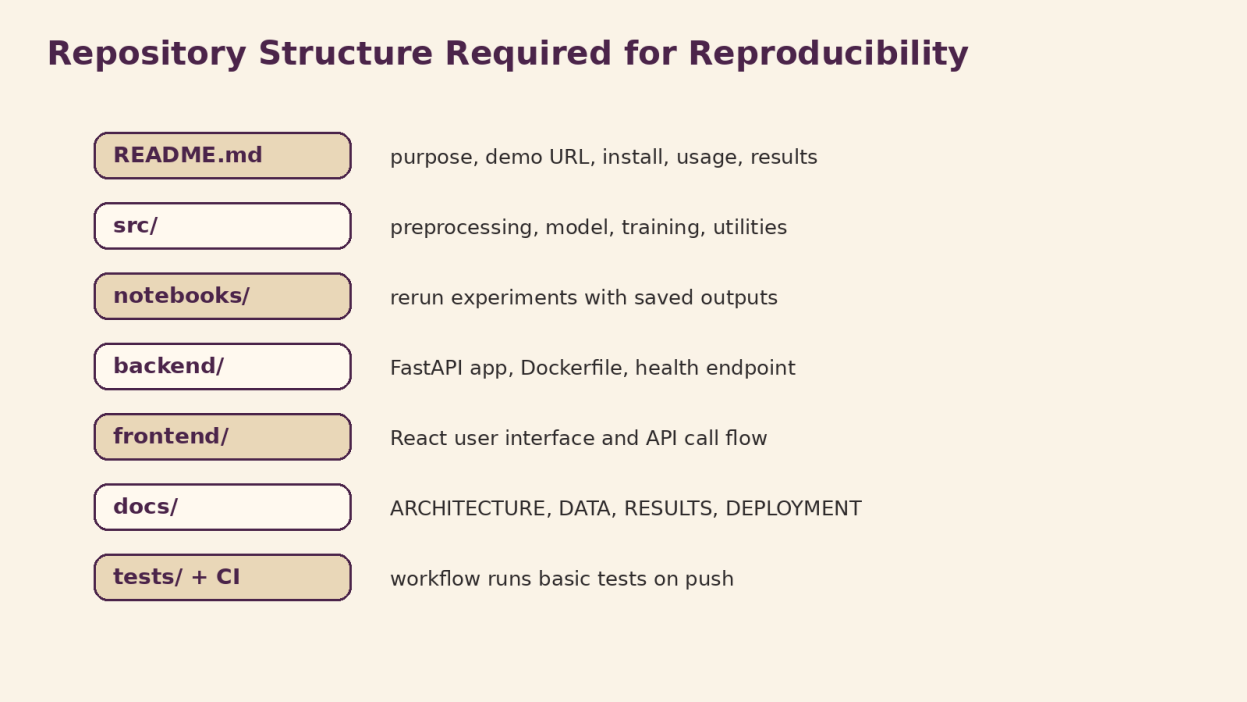


Figure 20. Required repository structure for reproducibility.

The final repository should be understandable to a stranger in under 30 minutes. The README should include project description, motivation, demo URL, screenshot or GIF, architecture diagram, installation steps, usage examples, results, citation and license. The docs folder should contain separate architecture, data, results and deployment documentation. Notebooks should be rerun with outputs so they can be reviewed on GitHub without execution.

12.2 Presentation Plan

Slide	Topic	Content
1	Title	Project, team, course and one line idea.
2	Problem	Why vulnerability detection matters.
3	Impact	Who benefits and what risk is reduced.
4	Related work	Why rules and plain models are limited.
5	Approach overview	CodeBERT + BiLSTM + VAE + HPO.
6 to 7	Architecture	Pipeline and component explanation.
8	Data and training	Dataset, preprocessing, imbalance handling.
9	Headline results	Final metrics and phase progression.
10	Key ablation	BiLSTM contribution and data quality insight.
11	Error analysis	Validation test gap and calibration honesty.
12	Demo	Live app or fallback video.
13	Limitations and future work	What remains and how to improve.
14	Team contribution	Who did what.
15	Q&A	Backup questions.

12.3 Demo Video Script

The demo video should be three to five minutes. Start with a 15 to 20 second introduction explaining who the team is and what SecureSense AI does. Then show the deployed app with at least three inputs: one clearly vulnerable buffer overflow or use after free

example, one safe C++ sample using std::string or bounds checked logic, and one edge case such as empty input or very long input. Spend 30 to 45 seconds on the architecture diagram and close with the GitHub repository and live demo URL.

13. CCP Rubric Compliance Map

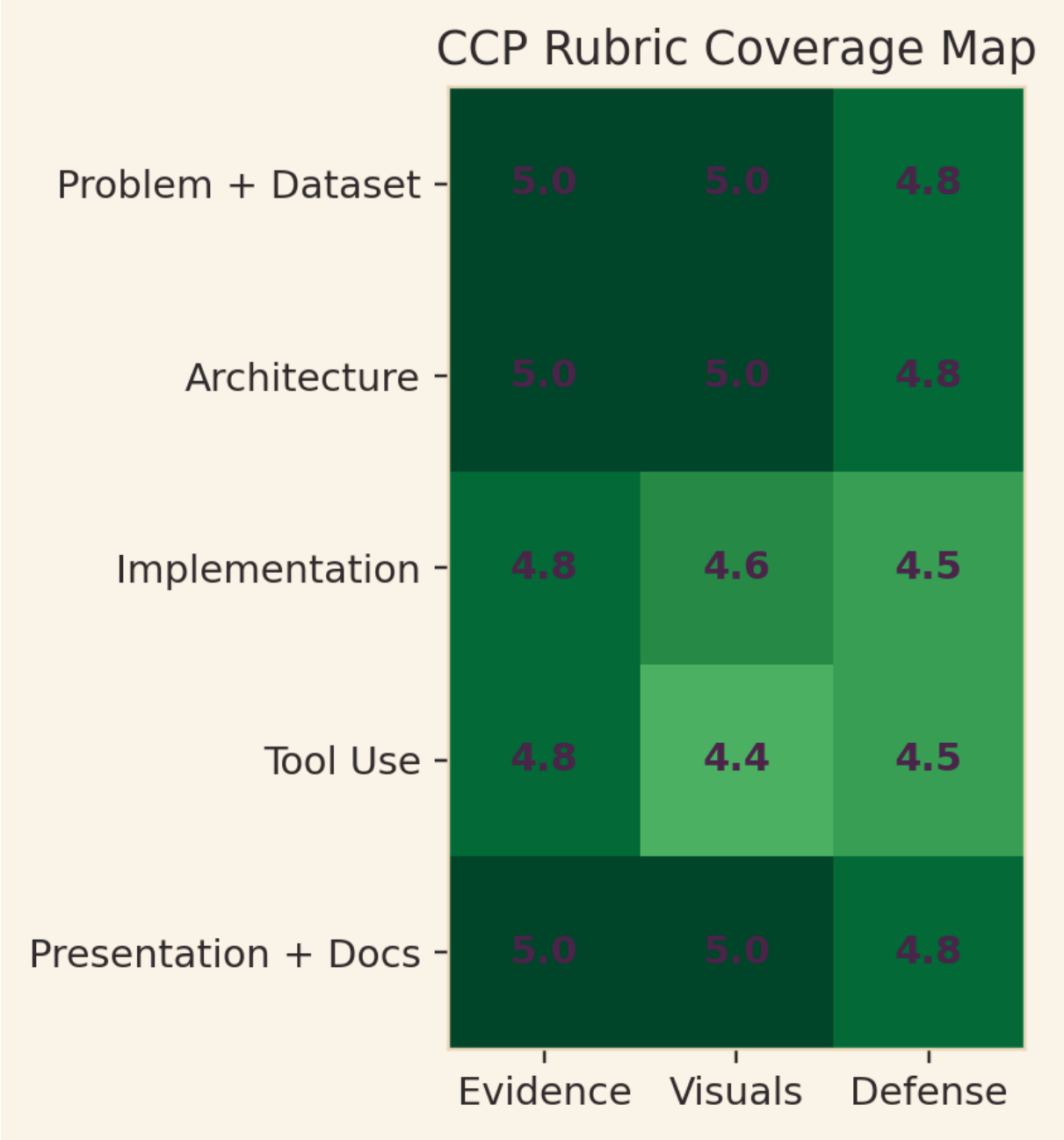


Figure 21. Rubric coverage map for CCP evaluation readiness.

Rubric criterion	Evidence in documentation	Readiness
Problem Identification and Dataset Understanding	Clear DL task, C/C++ sequence data, features, labels, preprocessing, augmentation, tokenization	Strong
Architecture Selection	Transformer + BiLSTM justified for code context and sequential flow	Strong
Logical Implementation	Workflow covers preprocessing, model building, training, evaluation and deployment	Strong
Tool Use	PyTorch, Transformers, Optuna, FastAPI, Docker, React and hosting platforms identified	Strong
Presentation and Documentation	Figures, training and validation evidence, evaluation metrics, ablations, limitations and	Strong

	demo plan	

100 marks strategy

The report should not rely on text only. It should show dataset charts, architecture diagrams, training workflow, HPO analysis, results charts, ablation chart, calibration plot, deployment diagram and requirement compliance evidence. This document includes all of those areas.

14. Conclusion and Future Work

SecureSense AI completes the full journey from a problem idea to a deployed deep learning system. The project identifies a real cybersecurity problem, builds an appropriate dataset pipeline, selects a suitable transformer plus sequence architecture, handles class imbalance through weighting and VAE augmentation, improves configuration through Bayesian search and evaluates the final model with ablations, robustness probes, calibration analysis and statistical intervals.

The most important project strength is integration. Each phase adds something meaningful. The most important project weakness is trust calibration. The final model is useful for educational triage and demonstration, but not ready for fully automated security decisions without calibration, label verification and broader testing.

14.1 Future Work

- Apply temperature scaling to reduce Expected Calibration Error before production triage.
- Run near duplicate detection and manually verify hard examples to reduce label noise.
- Replace 512 token truncation with sliding window or hierarchical encoding.
- Extend beyond C/C++ to Java, Python, Go and Rust through domain adaptive fine tuning.
- Add line level localization so developers can see the exact likely vulnerable line.
- Improve synthetic data by generating real code samples rather than only embeddings.
- Add continuous integration security scans and public model card documentation.

14.2 Final Defense Statement

The correct way to present the project is honest and confident: SecureSense AI is not a perfect replacement for expert security review. It is a deep learning based early warning tool that demonstrates strong code representation learning, a complete engineering workflow and a responsible understanding of limitations. That honesty is a strength during Q&A.

15. References

[1] Coverity Static Analysis, Synopsys Inc., industrial static analysis for C/C++ security.

[2] Li et al., VulDeePecker: A Deep Learning Based System for Multiclass Vulnerability Detection, NDSS 2018.

[3] Li et al., SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities, IEEE TDSC.

[4] Fu and Tantithamthavorn, LineVul: A Transformer Based Line Level Vulnerability Prediction, MSR 2022.

[5] Zhou et al., Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via GNNs, NeurIPS 2019.

[6] Feng et al., CodeBERT: A Pre Trained Model for Programming and Natural Languages, EMNLP Findings 2020.

[7] Guo et al., GraphCodeBERT: Pre Training Code Representations with Data Flow, ICLR 2021.

[8] Fan et al., A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries, MSR 2020.

[9] Chen et al., DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection, RAID 2023.

[10] Tihanyi et al., FormAI: Dataset for Formal AI Generated Code Security Evaluations, NeurIPS Workshop 2023.

[11] Hochreiter and Schmidhuber, Long Short Term Memory, Neural Computation 1997.

[12] Kingma and Welling, Auto Encoding Variational Bayes, ICLR 2014.

[13] Loshchilov and Hutter, Decoupled Weight Decay Regularization, ICLR 2019.

[14] Akiba et al., Optuna: A Next Generation Hyperparameter Optimization Framework, KDD 2019.

[15] MITRE Corporation, Common Weakness Enumeration CWE.

[16] SecureScan AI Project Repository, Ali Abbas Shah, Salman Tanveer and Hammad Ali, NIIT Lahore Spring 2026.

16. Appendices

Appendix A, Team Contribution Statement

Team member	Student ID	Primary contribution
Ali Abbas Shah	S2024cs012	SRS, literature review, architecture contribution, evaluation protocol, report sections and presentation preparation.
Salman Tanveer	S2024cs019	EDA, preprocessing, baseline, VAE, HPO, Hugging Face model artifact and results documentation.
Hammad Ali	S2024cs021	Training loop, transfer learning, ablation, efficiency benchmarking, frontend, Docker and deployment support.

Appendix B, Reproducibility Commands

```
git clone https://github.com/Salman1122334411/SecureScan-AI.git
cd SecureScan-AI
pip install -r requirements.txt
python phase5_evaluation.py --seed 42 --checkpoint checkpoints/phase4_final.pt
docker build -t securesense-api ./backend
docker run -p 8000:8000 securesense-api
```

Appendix C, Viva Preparation Questions

Question	Strong answer
Why CodeBERT?	Because the input is source code text and CodeBERT already understands code token patterns from pretraining.
Why BiLSTM after CodeBERT?	Because vulnerability meaning can depend on operation order, such as allocate, free, then use.
Why VAE?	To reduce minority class imbalance by generating vulnerable embeddings.
Why is confidence not fully trusted?	Because calibration analysis gives ECE 0.2937, so probability scores need correction.
Why deploy if limitations exist?	Because Phase 6 requires a usable academic prototype, and limitations are clearly stated for responsible use.